

CDC vertieft: SCD Type 1 vs. Type 2 mit AUTO CDC INTO

Section 3 · Data Transformation and Modelling · Quelle: Kurs 5, Kapitel 5–6

Change Data Capture (CDC) bezeichnet die Technik, Änderungen an einer Quelle – neue, geänderte oder gelöschte Datensätze – zu erkennen und in eine Zieltabelle zu übertragen, sodass diese jederzeit den aktuellen Stand der Quelle widerspiegelt. Wie mit diesen Änderungen historisch umgegangen wird, beschreiben die sogenannten **Slowly Changing Dimensions** (SCD). Kapitel 5 dieser Section hat bereits SCD Type 1 mit `AUTO CDC INTO` eingeführt; dieses Kapitel vertieft den Unterschied zu **SCD Type 2**, bei der die vollständige Änderungshistorie erhalten bleibt, und zeigt die dafür nötige Syntax anhand einer echten Kundendaten-Pipeline.

1. SCD Type 1 vs. SCD Type 2 im Vergleich

Bei **SCD Type 1** wird eine geänderte Zeile in der Zieltabelle einfach überschrieben – es existiert zu jedem Zeitpunkt nur die aktuelle Version eines Datensatzes, frühere Zustände gehen verloren. Das ist speicherschonend und einfach, eignet sich aber nicht, wenn die Historie einer Änderung selbst von Interesse ist. **SCD Type 2** hingegen bewahrt bei jeder Änderung die vorherige Version als abgeschlossenen historischen Datensatz und fügt eine neue Zeile mit den aktuellen Werten hinzu. Löschungen werden dabei nicht physisch entfernt, sondern als „Soft Delete“ markiert: Die Zeile bleibt bestehen, wird aber als inaktiv gekennzeichnet.

	SCD Type 1	SCD Type 2
Update-Verhalten	Bestehende Zeile wird überschrieben	Alte Zeile wird abgeschlossen, neue Zeile wird eingefügt
Historie	Nicht vorhanden	Vollständig nachvollziehbar
Löschungen	Zeile wird entfernt	Zeile bleibt erhalten, wird als inaktiv markiert
Speicherbedarf	Gering	Wächst mit jeder Änderung
Typischer Einsatz	Nur aktueller Stand relevant	Audit-Trails, Point-in-Time-Analysen, Compliance

2. AUTO CDC INTO: Syntax für SCD Type 2

Für beide SCD-Varianten stellt Apache Spark Declarative Pipelines denselben deklarativen Befehl bereit: `AUTO CDC INTO` (früher `APPLY CHANGES INTO` genannt – Syntax und Funktionsweise sind identisch geblieben, nur der Name wurde aktualisiert). Anstelle einer selbst geschriebenen `MERGE INTO`-Logik übernimmt die Pipeline automatisch die korrekte Reihenfolge der Ereignisse, das Auflösen von Updates gegen Inserts sowie – bei SCD Type 2 – das Schließen und Neuanlegen von Zeilen. Der Unterschied zu SCD Type 1 liegt allein in der Klausel `STORED AS SCD TYPE 2` am Ende der Anweisung.

Die vollständige Pipeline aus der Demo gliedert sich in drei Schritte: Zunächst wird die Bronze-Tabelle per Auto Loader aus JSON-Dateien befüllt, anschließend eine bereinigte Zwischentabelle mit Expectations erzeugt, und erst danach greift `AUTO CDC INTO` auf dieser bereinigten Quelle:

```

-- Schritt 1: Rohdaten inkrementell einlesen
CREATE OR REFRESH STREAMING TABLE sdp_cdc_1_bronze.customers_bronze_raw_demo
  COMMENT "Raw data from customers CDC feed"
AS
SELECT
  *,
  current_timestamp() processing_time,
  _metadata.file_name as source_file
FROM STREAM read_files(
  "${source}",
  format => "json");

-- Schritt 2: Typumwandlung und Data-Quality-Regeln auf Bronze-Ebene
CREATE STREAMING TABLE sdp_cdc_1_bronze.customers_bronze_clean_demo
(
  CONSTRAINT valid_id EXPECT (customer_id IS NOT NULL)
    ON VIOLATION FAIL UPDATE,
  CONSTRAINT valid_operation EXPECT (operation IS NOT NULL)
    ON VIOLATION DROP ROW,
  CONSTRAINT valid_email EXPECT (
    rlike(email, '^[a-zA-Z0-9_\\-\\.]+)@([a-zA-Z0-9_\\-\\.]+)\\.([a-zA-Z]{2,5})$')
    OR operation = "DELETE")
    ON VIOLATION DROP ROW
)
COMMENT "Clean raw bronze data and apply quality constraints"
AS
SELECT
  *,
  CAST(from_unixtime(timestamp) AS timestamp) AS timestamp_datetime
FROM STREAM sdp_cdc_1_bronze.customers_bronze_raw_demo;

-- Schritt 3: SCD Type 2 mit AUTO CDC INTO
CREATE OR REFRESH STREAMING TABLE sdp_cdc_2_silver.customers_silver_scd2_demo
  COMMENT 'SCD Type 2 Historical Customer Data';

CREATE FLOW customers_scd_type_2_flow AS
AUTO CDC INTO sdp_cdc_2_silver.customers_silver_scd2_demo
FROM STREAM sdp_cdc_1_bronze.customers_bronze_clean_demo
  KEYS (customer_id)
  APPLY AS DELETE WHEN operation = "DELETE"
  SEQUENCE BY timestamp_datetime
  COLUMNS * EXCEPT (timestamp, _rescued_data, operation)
  STORED AS SCD TYPE 2;

```

Die einzelnen Klauseln übernehmen klar getrennte Aufgaben: `KEYS (customer_id)` legt fest, anhand welcher Spalte(n) Datensätze aus Quelle und Ziel einander zugeordnet werden – auch zusammengesetzte Schlüssel aus mehreren Spalten sind möglich. `APPLY AS DELETE WHEN operation = "DELETE"` definiert, welche eingehenden Zeilen als Löschvorgang statt als Update

behandelt werden. `SEQUENCE BY timestamp_datetime` stellt sicher, dass Änderungen unabhängig von ihrer Ankunftsreihenfolge in der korrekt chronologischen Reihenfolge angewendet werden – entscheidend bei verspätet eintreffenden Ereignissen. `COLUMNS * EXCEPT (...)` schließt rein technische Spalten aus der Zieltabelle aus, und `STORED AS SCD TYPE 2` aktiviert schließlich die Historisierung.

3. Die Metadaten spalten `__START_AT` und `__END_AT`

Bei SCD Type 2 ergänzt `AUTO CDC INTO` die Zieltabelle automatisch um zwei Metadaten spalten: `__START_AT` hält fest, ab welchem Sequenzwert eine Zeile gültig war, `__END_AT` markiert das Ende dieser Gültigkeit. Ein `NULL`-Wert in `__END_AT` kennzeichnet die aktuell gültige Version eines Datensatzes; ein gesetzter Wert zeigt eine historische, nicht mehr aktive Zeile an. Bei einer Löschung wird kein neuer Datensatz angelegt – stattdessen erhält die zuletzt aktive Zeile lediglich ein `__END_AT`, während der ursprüngliche Inhalt erhalten bleibt.

SCD Type 2 (Historical Tracking/Versioning)

- When a record changes:
 - The **old record is preserved** with an additional column indicating its validity period (start date, end date, or a current flag) and a **new record is inserted** with the updated information.
 - When a **record is marked as deleted**, the record is kept and a column indicates the record is **inactive**.
- Used when **historical data is important**, and the system needs to track **how attributes change over time** (like tracking changes in a customer's address or status).

Example customers target table with SCD Type 2

CustomerID	Name	Address	ProcessDate	__START_AT	__END_AT
1	Peter	123 Main St.	5/20/2025	5/20/2025	null
1	Peter	1 Blue Rd.	5/1/2025	5/1/2025	5/20/2025
2	Samarth	22 Front St	5/1/2025	5/1/2025	5/20/2025
3	Kostas	100 Athens Dr.	5/20/2025	5/20/2025	null

Columns indicate **active** and **inactive** rows

Null values indicate the **current** row

Non null values indicate **inactive** (historic) rows

© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Für Kunde Peter existieren nach mehreren Adressänderungen zwei Zeilen: die historische mit gesetztem `__END_AT` und die aktuelle mit `__END_AT = NULL`. Der gelöschte Kunde Samarth bleibt als inaktive Zeile erhalten.

Auf dieser Grundlage lassen sich in der Gold-Schicht gezielt unterschiedliche Sichten bilden. Die folgenden zwei Materialized Views aus der Demo-Pipeline zeigen das Muster: eine View mit ausschließlich aktuell aktiven Kunden, sowie eine zweite View, die per `MAX_BY` gezielt die zuletzt aktive Version aller inzwischen gelöschten Kunden zusammenstellt:

```
-- Aktuell aktive Kunden: __END_AT ist NULL
CREATE OR REFRESH MATERIALIZED VIEW sdp_cdc_3_gold.current_customers_gold_demo
COMMENT "Current updated list of active customers"
AS
SELECT
    * EXCEPT (processing_time),
    current_timestamp() updated_at
FROM sdp_cdc_2_silver.customers_silver_scd2_demo
WHERE `__END_AT` IS NULL;

-- Gelöschte Kunden: letzte bekannte Version je Kunde, __END_AT vorhanden
CREATE OR REFRESH MATERIALIZED VIEW sdp_cdc_3_gold.removed_customers_gold_demo AS
SELECT
    customer_id,
    MAX_BY(name, __START_AT)      AS name,
    MAX_BY(address, __START_AT)   AS address,
    MAX_BY(__START_AT, __START_AT) AS __START_AT,
    MAX_BY(__END_AT, __START_AT)  AS __END_AT
FROM sdp_cdc_2_silver.customers_silver_scd2_demo
GROUP BY customer_id
HAVING MAX_BY(__END_AT, __START_AT) IS NOT NULL;
```

Damit lässt sich mit vergleichsweise wenig Code eine vollständig historisierte Kundendimension abbilden – inklusive sauberer Trennung von aktivem Bestand und Löschhistorie, ohne dass eine einzige Zeile manueller `MERGE` -Logik geschrieben werden musste.

Ergänzt aus der offiziellen Databricks-Dokumentation: Die aktuelle Syntaxreferenz zu `AUTO CDC INTO` führt neben `STORED AS SCD TYPE 1` und `SCD TYPE 2` inzwischen auch die Option `BITEMPORAL` sowie zusätzliche Klauseln wie `IGNORE NULL UPDATES`, `APPLY AS TRUNCATE WHEN` und `TRACK HISTORY ON` für eine noch feinere Steuerung, welche Spalten überhaupt eine Historisierung auslösen. Für SCD-Type-2-Zieltabellen gilt außerdem: Werden `__START_AT` und `__END_AT` beim expliziten Anlegen des Tabellenschemas mit angegeben, müssen sie denselben Datentyp wie die `SEQUENCE BY` -Spalte besitzen.

Quelle: [AUTO CDC INTO \(pipelines\) – Databricks-Dokumentation](#)